

Tutorial — Centralizador de Logs com Loki + Alloy + Grafana

1. Identificação do documento

Item	Informação
Ambiente	Homelab
Servidor/VM	observabilidade
Sistema operacional	Rocky Linux 9
IP	192.168.31.17
DNS interno da VM	observabilidade.jmsalles.homelab.br
URL publicada	logs.jmsalles.homelab.br
Stack	Loki + Alloy + Grafana
Método de instalação	Docker Compose
SELinux	Desabilitado
Firewall local	Desabilitado
Diretório base	/opt/observability
Retenção inicial dos logs	15 dias
Usuário Grafana	admin
Senha padrão homelab	4796@@Eumesmo1234567

2. Objetivo

Implantar uma stack centralizada de logs no homelab usando:

Loki + Alloy + Grafana

O objetivo é permitir a centralização, consulta e investigação de logs de:

Servidores Linux
Docker / Portainer
Kubernetes
Nginx reverse proxy
Aplicações em containers
Serviços do homelab

Nesta primeira etapa, o foco será instalar a base central na VM **observabilidade**, validar o funcionamento local e publicar o acesso via Nginx usando:

```
https://logs.jmsalles.homelab.br
```

3. Escopo desta etapa

Esta etapa contempla:

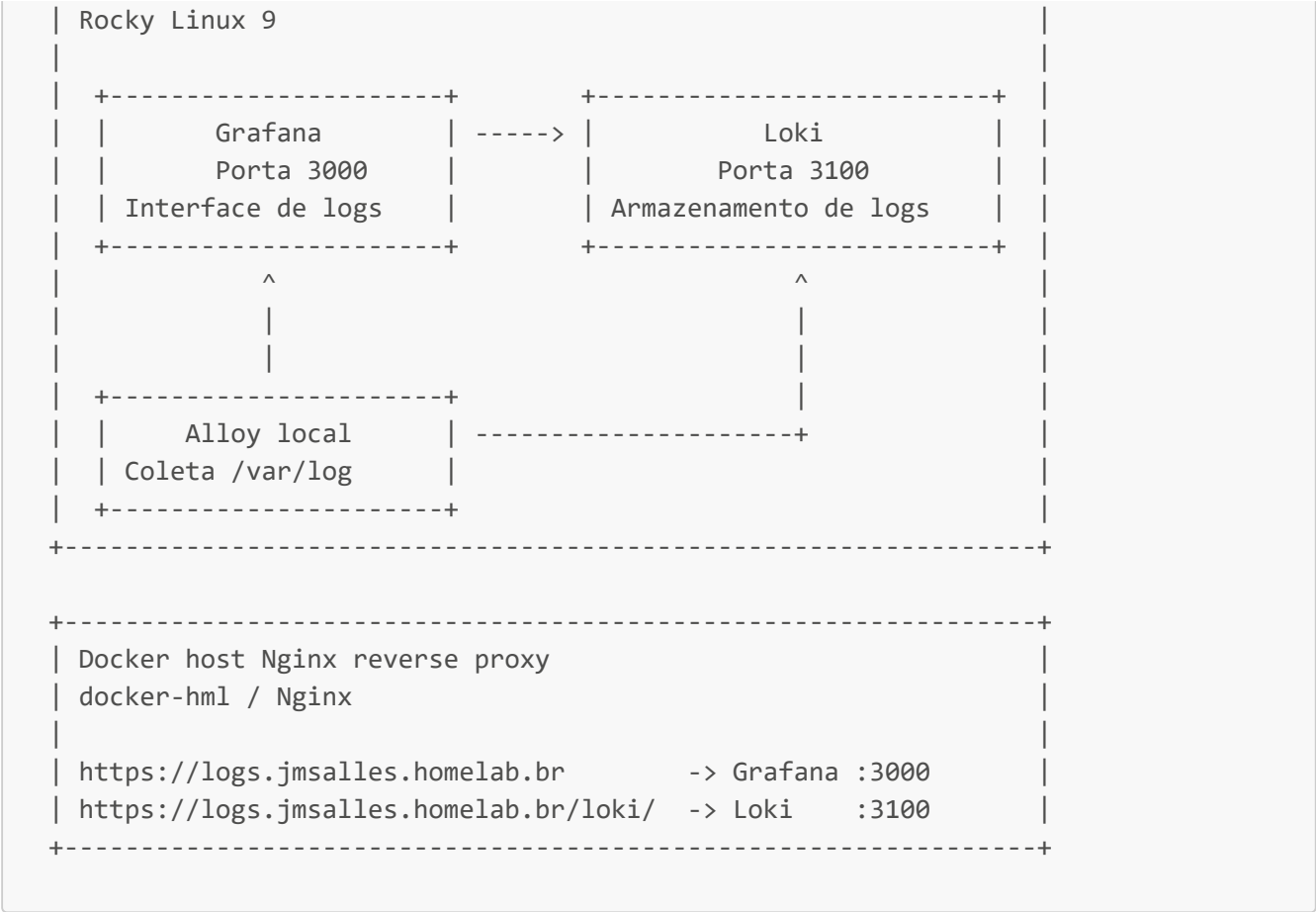
1. Validação da VM observabilidade
2. Desabilitação de SELinux
3. Desabilitação do firewalld
4. Instalação do Docker Engine
5. Instalação do Docker Compose Plugin
6. Criação da estrutura /opt/observability
7. Configuração do Loki
8. Configuração do Grafana
9. Configuração do Alloy local
10. Subida da stack via docker-compose.yml
11. Testes de Loki
12. Testes de Grafana
13. Testes de coleta de logs locais
14. Backup das configurações da stack
15. Criação do DNS logs.jmsalles.homelab.br
16. Publicação via Nginx reverse proxy
17. Comandos operacionais
18. Troubleshooting
19. Próximas fases

Esta etapa ainda **não** contempla:

Coleta de logs de outros servidores Linux
Coleta de logs do Docker/Portainer em outros hosts
Coleta de logs do Kubernetes
Dashboards avançados
Alertas no Grafana
Integração com Zabbix

4. Arquitetura proposta

```
+-----+
| VM observabilidade |
| IP: 192.168.31.17  |
| DNS: observabilidade.jmsalles.homelab.br |
```



5. Papel de cada componente

Componente	Função
Loki	Armazenar e consultar logs
Grafana	Interface web para busca, visualização e análise
Alloy	Coletor de logs
Docker Compose	Orquestrar os containers da stack
Nginx	Publicar o Grafana com HTTPS interno
Zabbix	Continua sendo o monitoramento principal de disponibilidade e triggers

6. Padrão definido para este ambiente

6.1. Nome da VM

observabilidade

6.2. IP da VM

```
192.168.31.17
```

6.3. DNS interno da VM

```
observabilidade.jmsalles.homelab.br
```

6.4. DNS publicado no Nginx

```
logs.jmsalles.homelab.br
```

6.5. URL publicada do Grafana

```
https://logs.jmsalles.homelab.br
```

6.6. URL publicada do Loki

```
https://logs.jmsalles.homelab.br/loki/
```

6.7. URL interna do Grafana

```
http://192.168.31.17:3000
```

6.8. URL interna do Loki

```
http://192.168.31.17:3100
```

6.9. Senha padrão da stack

```
4796@@Eumesmo1234567
```

7. Acesso inicial à VM

Acessar a VM:

```
ssh jmsalles@192.168.31.17
```

Elevar para root:

```
sudo -i
```

Validar hostname:

```
hostname
```

Validar FQDN:

```
hostname -f
```

Validar sistema operacional:

```
cat /etc/redhat-release
```

Validar IP:

```
ip -br addr
```

Resultado esperado:

```
observabilidade  
Rocky Linux 9.x  
192.168.31.17
```

Se o hostname ainda não estiver correto:

```
hostnamectl set-hostname observabilidade
```

Validar novamente:

```
hostname  
hostname -f
```

8. Validação de rede e DNS da VM

Validar resolução do DNS interno:

```
getent hosts observabilidade.jmsalles.homelab.br
```

Resultado esperado:

```
192.168.31.17 observabilidade.jmsalles.homelab.br
```

Validar gateway da rede:

```
ping -c 3 192.168.31.2
```

Validar DNS interno/AD:

```
ping -c 3 192.168.31.24
```

Validar resolução externa:

```
ping -c 3 google.com
```

Validar rota padrão:

```
ip route
```

Resultado esperado:

```
default via 192.168.31.2
```

9. Atualização inicial do sistema

Atualizar pacotes:

```
dnf update -y
```

Se houver atualização de kernel, reiniciar:

```
reboot
```

Após o reboot, acessar novamente:

```
ssh jmsalles@192.168.31.17  
sudo -i
```

Validar uptime:

```
uptime
```

Validar kernel:

```
uname -r
```

10. Desabilitar SELinux

Verificar status atual:

```
getenforce
```

```
sestatus
```

Se estiver **Enforcing**, desabilitar temporariamente:

```
setenforce 0
```

Desabilitar permanentemente:

```
sed -i 's/^SELINUX=.*SELINUX=disabled/' /etc/selinux/config
```

Validar arquivo:

```
grep ^SELINUX= /etc/selinux/config
```

Resultado esperado:

```
SELINUX=disabled
```

Reiniciar a VM:

```
reboot
```

Após voltar:

```
ssh jmsalles@192.168.31.17  
sudo -i
```

Validar:

```
getenforce
```

Resultado esperado:

```
Disabled
```

Validar também:

```
sestatus
```

11. Desabilitar firewall local

Parar e desabilitar:

```
systemctl disable --now firewalld
```

Validar status:

```
systemctl status firewalld --no-pager
```

Resultado esperado:

```
inactive  
disabled
```

Validar:

```
systemctl is-enabled firewalld
```

Resultado esperado:

```
disabled
```

12. Instalar pacotes básicos

Instalar ferramentas úteis:

```
dnf install -y vim curl wget tar unzip net-tools bind-utils jq dnf-plugins-core  
yum-utils tree
```

Validar comandos básicos:

```
which curl  
which wget  
which jq
```

```
which nslookup  
which dig  
which tree
```

13. Instalar Docker Engine

Adicionar o repositório oficial do Docker para RHEL/Rocky:

```
dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-  
ce.repo
```

Atualizar metadados:

```
dnf makecache
```

Instalar Docker Engine e plugins:

```
dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin  
docker-compose-plugin
```

Habilitar Docker:

```
systemctl enable --now docker
```

Validar status:

```
systemctl status docker --no-pager
```

Validar versão do Docker:

```
docker version
```

Validar Docker Compose Plugin:

```
docker compose version
```

14. Teste básico do Docker

Executar container de teste:

```
docker run --rm hello-world
```

Resultado esperado:

```
Hello from Docker!
```

Validar containers:

```
docker ps -a
```

Validar imagens:

```
docker images
```

15. Configurar rotação de logs do Docker

Essa etapa evita que os logs dos containers cresçam indefinidamente em disco.

Criar arquivo:

```
vim /etc/docker/daemon.json
```

Conteúdo:

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "50m",
    "max-file": "5"
  }
}
```

Reiniciar Docker:

```
systemctl restart docker
```

Validar Docker:

```
systemctl status docker --no-pager
```

Validar logging driver:

```
docker info | grep -i "Logging Driver"
```

Resultado esperado:

```
Logging Driver: json-file
```

Validar configuração completa:

```
docker info | egrep -i 'Logging Driver|Cgroup|Storage Driver'
```

16. Criar estrutura da stack

Diretório base:

```
/opt/observability
```

Criar diretórios usando brace expansion:

```
mkdir -p  
/opt/observability/{loki,grafana/provisioning/datasources,grafana/data,alloy,data/  
loki,backup}
```

Validar:

```
tree -d /opt/observability
```

Estrutura esperada:

```
/opt/observability
├── alloy
├── backup
├── data
│   ├── loki
├── grafana
│   ├── data
│   ├── provisioning
│   └── datasources
└── loki
```

Ajustar permissões do Grafana:

```
chown -R 472:472 /opt/observability/grafana
```

Ajustar permissões do Loki:

```
chown -R 10001:10001 /opt/observability/data/loki
```

Validar:

```
ls -lah /opt/observability
ls -lah /opt/observability/grafana
ls -lah /opt/observability/data
```

17. Criar arquivo `.env`

Criar o arquivo de variáveis da stack:

```
vim /opt/observability/.env
```

Conteúdo:

```
GRAFANA_ADMIN_USER=admin  
GRAFANA_ADMIN_PASSWORD=4796@@Eumesmo1234567
```

Ajustar permissão:

```
chmod 600 /opt/observability/.env
```

Validar:

```
ls -lh /opt/observability/.env
```

Resultado esperado:

```
-rw----- root root /opt/observability/.env
```

18. Criar configuração do Loki

Criar arquivo:

```
vim /opt/observability/loki/loki-config.yml
```

Conteúdo:

```
auth_enabled: false  
  
server:  
  http_listen_port: 3100  
  grpc_listen_port: 9095  
  
common:  
  path_prefix: /loki  
  replication_factor: 1  
  ring:  
    kvstore:  
      store: inmemory  
  storage:  
    filesystem:  
      chunks_directory: /loki/chunks  
      rules_directory: /loki/rules
```

```
schema_config:
  configs:
    - from: 2024-01-01
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h

limits_config:
  retention_period: 360h
  allow_structured_metadata: true
  volume_enabled: true

compactor:
  working_directory: /loki/compactor
  compaction_interval: 10m
  retention_enabled: true
  retention_delete_delay: 2h
  delete_request_store: filesystem
```

Explicação dos principais parâmetros:

Parâmetro	Função
auth_enabled: false	Desabilita autenticação/multitenancy no Loki
http_listen_port: 3100	Porta HTTP do Loki
schema: v13	Schema usado pelo Loki
object_store: filesystem	Armazenamento local em filesystem
retention_period: 360h	Retenção de 15 dias
retention_enabled: true	Habilita retenção via compactor
delete_request_store: filesystem	Armazena requisições de deleção em filesystem

Validar arquivo:

```
cat /opt/observability/loki/loki-config.yml
```

19. Criar datasource do Grafana

Esse arquivo faz o Grafana já subir com o Loki configurado como datasource.

Criar arquivo:

```
vim /opt/observability/grafana/provisioning/datasources/loki.yml
```

Conteúdo:

```
apiVersion: 1

datasources:
- name: Loki
  type: loki
  access: proxy
  url: http://loki:3100
  isDefault: true
  editable: true
```

Validar:

```
cat /opt/observability/grafana/provisioning/datasources/loki.yml
```

20. Criar configuração do Alloy local

O Alloy será responsável por coletar os logs locais da própria VM **observabilidade** e enviar para o Loki.

Criar arquivo:

```
vim /opt/observability/alloy/config.alloy
```

Conteúdo:

```
logging {
  level = "info"
  format = "logfmt"
}

loki.write "default" {
  endpoint {
    url = "http://loki:3100/loki/api/v1/push"
  }
}

local.file_match "linux_logs" {
  path_targets = [
    {
```

```
    "__path__" = "/var/log/messages",
    "job" = "linux-logs",
    "host" = "observabilidade",
    "fqdn" = "observabilidade.jmsalles.homelab.br",
    "log_type" = "messages",
  },
  {
    "__path__" = "/var/log/secure",
    "job" = "linux-logs",
    "host" = "observabilidade",
    "fqdn" = "observabilidade.jmsalles.homelab.br",
    "log_type" = "secure",
  },
  {
    "__path__" = "/var/log/cron",
    "job" = "linux-logs",
    "host" = "observabilidade",
    "fqdn" = "observabilidade.jmsalles.homelab.br",
    "log_type" = "cron",
  },
  {
    "__path__" = "/var/log/dnf.log",
    "job" = "linux-logs",
    "host" = "observabilidade",
    "fqdn" = "observabilidade.jmsalles.homelab.br",
    "log_type" = "dnf",
  },
]
}

loki.source.file "linux_logs" {
  targets      = local.file_match.linux_logs.targets
  forward_to   = [loki.process.linux.receiver]
}

loki.process "linux" {
  stage.static_labels {
    values = {
      env      = "homelab",
      source   = "linux",
      role     = "observability",
    }
  }

  forward_to = [loki.write.default.receiver]
}
```

Validar arquivo:

```
cat /opt/observability/alloy/config.alloy
```

21. Criar `docker-compose.yml`

Criar arquivo:

```
vim /opt/observability/docker-compose.yml
```

Conteúdo:

```
services:
  loki:
    image: grafana/loki:latest
    container_name: loki
    command: -config.file=/etc/loki/loki-config.yml
    ports:
      - "3100:3100"
    volumes:
      - ./loki/loki-config.yml:/etc/loki/loki-config.yml:ro
      - ./data/loki:/loki
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    env_file:
      - .env
    environment:
      GF_SECURITY_ADMIN_USER: "${GRAFANA_ADMIN_USER}"
      GF_SECURITY_ADMIN_PASSWORD: "${GRAFANA_ADMIN_PASSWORD}"
      GF_USERS_ALLOW_SIGN_UP: "false"
      GF_SERVER_DOMAIN: "logs.jmsalles.homelab.br"
      GF_SERVER_ROOT_URL: "https://logs.jmsalles.homelab.br/"
      GF_SERVER_SERVE_FROM_SUB_PATH: "false"
    volumes:
      - ./grafana/data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
    depends_on:
      - loki
    restart: unless-stopped

  alloy-local:
    image: grafana/alloy:latest
    container_name: alloy-local
    command:
      - run
      - --server.http.listen-addr=0.0.0.0:12345
      - /etc/alloy/config.alloy
    volumes:
```

```
- ./alloy/config.alloy:/etc/alloy/config.alloy:ro
- /var/log:/var/log:ro
depends_on:
- loki
restart: unless-stopped
```

Observação:

Como SELinux estará desabilitado, não usamos :Z nos volumes.
Como firewalld estará desabilitado, não usamos firewall-cmd.
O Grafana já foi ajustado para funcionar por trás do Nginx em:
<https://logs.jmsalles.homelab.br>

Validar arquivo:

```
cat /opt/observability/docker-compose.yml
```

Validar sintaxe do Compose:

```
cd /opt/observability
docker compose config
```

Se o comando retornar a configuração renderizada sem erro, o arquivo está válido.

22. Subir a stack

Entrar no diretório:

```
cd /opt/observability
```

Baixar imagens:

```
docker compose pull
```

Subir containers:

```
docker compose up -d
```

Validar:

```
docker compose ps
```

Resultado esperado:

NAME	IMAGE	STATUS
grafana	grafana/grafana	Up
loki	grafana/loki	Up
alloy-local	grafana/alloy	Up

Validar containers pelo Docker:

```
docker ps
```

23. Verificar logs dos containers

Logs do Loki:

```
cd /opt/observability  
docker compose logs -f loki
```

Logs do Grafana:

```
cd /opt/observability  
docker compose logs -f grafana
```

Logs do Alloy:

```
cd /opt/observability  
docker compose logs -f alloy-local
```

Para sair do modo follow:

```
CTRL + C
```

24. Validar portas abertas na VM

Como o firewall local está desabilitado, basta validar se os containers estão escutando.

```
ss -lntp | egrep '3000|3100|12345'
```

Resultado esperado:

```
0.0.0.0:3000 Grafana
0.0.0.0:3100 Loki
0.0.0.0:12345 Alloy
```

Portas usadas:

Porta	Serviço
3000/tcp	Grafana
3100/tcp	Loki
12345/tcp	Alloy local

25. Validar Loki

Teste local:

```
curl -s http://127.0.0.1:3100/ready
```

Resultado esperado:

```
ready
```

Teste pelo IP:

```
curl -s http://192.168.31.17:3100/ready
```

Resultado esperado:

```
ready
```

Teste pelo DNS interno:

```
curl -s http://observabilidade.jmsalles.homelab.br:3100/ready
```

Resultado esperado:

```
ready
```

Validar endpoint de métricas do Loki:

```
curl -s http://127.0.0.1:3100/metrics | head
```

Validar labels:

```
curl -s "http://127.0.0.1:3100/loki/api/v1/labels" | jq
```

26. Acessar Grafana internamente

Antes de configurar o Nginx, validar o Grafana diretamente pela VM:

```
http://192.168.31.17:3000
```

Credenciais:

```
Usuário: admin  
Senha: 4796@@Eumesmo1234567
```

27. Validar datasource Loki no Grafana

No Grafana:

```
Connections
Data sources
Loki
Save & test
```

Resultado esperado:

```
Data source connected and labels found
```

Depois acessar:

```
Explore
Data source: Loki
```

Consulta inicial:

```
{host="observabilidade"}
```

Se ainda não aparecer nada, gerar um log manual:

```
logger "VALIDACAO LOKI ALLOY GRAFANA - observabilidade - $(date)"
```

Depois consultar no Grafana:

```
{host="observabilidade"} |= "VALIDACAO LOKI"
```

28. Validar logs por linha de comando

Consultar labels:

```
curl -s "http://127.0.0.1:3100/loki/api/v1/labels" | jq
```

Consultar valores da label **host**:

```
curl -s "http://127.0.0.1:3100/loki/api/v1/label/host/values" | jq
```

Resultado esperado:

```
[  
  "observabilidade"  
]
```

Consultar valores da label `log_type`:

```
curl -s "http://127.0.0.1:3100/loki/api/v1/label/log_type/values" | jq
```

Resultado esperado:

```
[  
  "cron",  
  "dnf",  
  "messages",  
  "secure"  
]
```

Consultar valores da label `job`:

```
curl -s "http://127.0.0.1:3100/loki/api/v1/label/job/values" | jq
```

Resultado esperado:

```
[  
  "linux-logs"  
]
```

29. Consultas úteis no Grafana

Todos os logs da VM observabilidade:

```
{host="observabilidade"}
```

Logs do sistema:

```
{host="observabilidade", log_type="messages"}
```

Logs de segurança:

```
{host="observabilidade", log_type="secure"}
```

Logs de cron:

```
{host="observabilidade", log_type="cron"}
```

Logs do DNF:

```
{host="observabilidade", log_type="dnf"}
```

Buscar erros:

```
{host="observabilidade"} |= "error"
```

Buscar falhas:

```
{host="observabilidade"} |= "failed"
```

Buscar uso de sudo:

```
{host="observabilidade", log_type="secure"} |= "sudo"
```

Buscar login SSH:

```
{host="observabilidade", log_type="secure"} |= "sshd"
```

Buscar mensagem de validação:

```
{host="observabilidade"} |= "VALIDACAO"
```

30. Validação completa da stack

Executar na VM:

```
hostname
hostname -f
ip -br addr
df -hT
free -h
docker version
docker compose version
docker compose -f /opt/observability/docker-compose.yml ps
curl -s http://127.0.0.1:3100/ready
curl -s http://observabilidade.jmsalles.homelab.br:3100/ready
```

Resultado esperado:

```
Hostname: observabilidade
IP: 192.168.31.17
Docker: funcionando
Docker Compose: funcionando
Loki: ready
Grafana: acessível na porta 3000
Alloy: coletando logs locais
```

Gerar evidência de log:

```
logger "EVIDENCIA FINAL - LOKI ALLOY GRAFANA FUNCIONANDO - observabilidade - $(date)"
```

Consultar no Grafana:

```
{host="observabilidade"} |= "EVIDENCIA FINAL"
```

31. Comandos operacionais

Acessar diretório da stack:

```
cd /opt/observability
```

Ver containers:

```
docker compose ps
```

Subir stack:

```
docker compose up -d
```

Parar stack:

```
docker compose down
```

Reiniciar stack:

```
docker compose restart
```

Reiniciar apenas Loki:

```
docker compose restart loki
```

Reiniciar apenas Grafana:

```
docker compose restart grafana
```

Reiniciar apenas Alloy:

```
docker compose restart alloy-local
```

Logs do Loki:

```
docker compose logs -f loki
```

Logs do Grafana:

```
docker compose logs -f grafana
```

Logs do Alloy:

```
docker compose logs -f alloy-local
```

Ver consumo dos containers:

```
docker stats
```

Ver uso de disco do Loki:

```
du -sh /opt/observability/data/loki
```

Ver uso geral da VM:

```
df -hT  
free -h  
uptime
```

32. Atualização futura da stack

Para atualizar as imagens:

```
cd /opt/observability  
docker compose pull  
docker compose up -d
```

Validar:

```
docker compose ps
```

Ver logs após atualização:

```
docker compose logs --tail=100 loki
docker compose logs --tail=100 grafana
docker compose logs --tail=100 alloy-local
```

33. Backup das configurações

Este backup salva somente as configurações da stack, não os dados dos logs.

33.1. Criar diretório

```
mkdir -p /opt/backup-observability
```

33.2. Criar script

```
vim /usr/local/sbin/backup-observability-config.sh
```

Conteúdo:

```
#!/bin/bash

DATA=$(date +%F_%H-%M-%S)
DESTINO="/opt/backup-observability"
ARQUIVO="${DESTINO}/observability-config-${DATA}.tar.gz"
LOG="/var/log/backup-observability-config.log"

mkdir -p "$DESTINO"

echo "[$(date '+%F %T')] Iniciando backup das configuracoes da stack
observability" >> "$LOG"

tar -czf "$ARQUIVO" \
  /opt/observability/docker-compose.yml \
  /opt/observability/.env \
  /opt/observability/loki \
  /opt/observability/grafana/provisioning \
  /opt/observability/alloy
```

```
if [ $? -eq 0 ]; then
    echo "[$(date '+%F %T')] Backup criado com sucesso: $ARQUIVO" >> "$LOG"
else
    echo "[$(date '+%F %T')] ERRO ao criar backup" >> "$LOG"
    exit 1
fi

find "$DESTINO" -type f -name "observability-config-*.tar.gz" -mtime +30 -delete

echo "[$(date '+%F %T')] Rotina finalizada" >> "$LOG"
```

33.3. Permissão

```
chmod +x /usr/local/sbin/backup-observability-config.sh
```

33.4. Testar backup

```
/usr/local/sbin/backup-observability-config.sh
```

33.5. Validar arquivo gerado

```
ls -lh /opt/backup-observability
```

33.6. Validar log

```
tail -n 50 /var/log/backup-observability-config.log
```

33.7. Agendar no cron

Editar crontab do root:

```
crontab -e
```

Adicionar:

```
15 1 * * * /usr/local/sbin/backup-observability-config.sh >> /var/log/backup-observability-config.log 2>&1
```

Validar crontab:

```
crontab -l
```

34. Backup opcional dos dados do Loki

Para homelab, inicialmente eu não recomendo backup completo dos dados do Loki, porque logs são dados volumosos e temporários. A retenção inicial é de 15 dias.

Mas, se quiser backup manual dos dados, pode usar:

```
cd /opt/observability
docker compose stop loki
tar -czf /opt/backup-observability/loki-data-$(date +%F_%H-%M-%S).tar.gz
/opt/observability/data/loki
docker compose start loki
```

Validar:

```
ls -lh /opt/backup-observability
```

35. Procedimento de parada controlada

Parar stack:

```
cd /opt/observability
docker compose down
```

Validar:

```
docker ps
```

Confirmar que portas pararam:

```
ss -lntp | egrep '3000|3100|12345'
```

Se não retornar nada, as portas foram liberadas.

36. Procedimento de subida controlada

Subir stack:

```
cd /opt/observability  
docker compose up -d
```

Validar:

```
docker compose ps
```

Validar Loki:

```
curl -s http://127.0.0.1:3100/ready
```

Validar Grafana:

```
http://192.168.31.17:3000
```

37. Procedimento de rollback simples

Caso uma alteração quebre a stack, restaurar backup de configuração.

37.1. Parar stack

```
cd /opt/observability  
docker compose down
```

37.2. Listar backups

```
ls -lh /opt/backup-observability
```

37.3. Restaurar backup

Exemplo:

```
tar -xzf /opt/backup-observability/observability-config-YYYY-MM-DD_HH-MM-SS.tar.gz -C /
```

37.4. Subir stack novamente

```
cd /opt/observability  
docker compose up -d
```

37.5. Validar

```
docker compose ps  
curl -s http://127.0.0.1:3100/ready
```

38. Criar DNS e publicar via Nginx reverse proxy

Nesta etapa vamos publicar o Grafana através do Nginx reverse proxy no padrão que você já usa no homelab.

A URL final será:

```
https://logs.jmsalles.homelab.br
```

E o proxy fará encaminhamento para:

```
Grafana: http://192.168.31.17:3000  
Loki:    http://192.168.31.17:3100
```

38.1. Criar entrada DNS

No seu DNS interno/AD, criar a entrada:

```
Nome: logs  
FQDN: logs.jmsalles.homelab.br  
Tipo: A  
IP: 192.168.31.37
```

Explicação:

```
192.168.31.37 = host do Nginx reverse proxy  
192.168.31.17 = VM observabilidade
```

O navegador acessa:

```
https://logs.jmsalles.homelab.br
```

O DNS aponta para o Nginx:

```
logs.jmsalles.homelab.br -> 192.168.31.37
```

E o Nginx encaminha para a VM:

```
192.168.31.17:3000
```

38.2. Validar DNS a partir da sua estação

No Windows:

```
nslookup logs.jmsalles.homelab.br
```

Resultado esperado:

```
Name: logs.jmsalles.homelab.br  
Address: 192.168.31.37
```

No Linux:

```
getent hosts logs.jmsalles.homelab.br
```

Resultado esperado:

```
192.168.31.37 logs.jmsalles.homelab.br
```

Também pode validar com:

```
dig logs.jmsalles.homelab.br
```

38.3. Acessar host do Nginx

Acessar o host onde está o seu Nginx reverse proxy:

```
ssh jmsalles@docker-hml.jmsalles.homelab.br
```

Elevar para root, se necessário:

```
sudo -i
```

Ir para o diretório do Nginx:

```
cd /home/jmsalles/nginx
```

Validar estrutura:

```
ls -lah
```

Validar diretório de configurações:

```
ls -lah /home/jmsalles/nginx/conf.d
```

Validar certificados:

```
ls -lah /home/jmsalles/nginx/ssl
```

Esperado encontrar algo parecido com:

```
jmsalles.homelab.br.crt  
jmsalles.homelab.br.key
```

38.4. Criar arquivo de configuração do Nginx

Criar o arquivo:

```
vim /home/jmsalles/nginx/conf.d/logs.jmsalles.homelab.br.conf
```

Conteúdo:

```
server {  
    listen 80;  
    server_name logs.jmsalles.homelab.br;  
  
    return 301 https://$host$request_uri;  
}  
  
server {  
    listen 443 ssl;  
    http2 on;  
  
    server_name logs.jmsalles.homelab.br;  
  
    ssl_certificate      /etc/nginx/ssl/jmsalles.homelab.br.crt;  
    ssl_certificate_key  /etc/nginx/ssl/jmsalles.homelab.br.key;  
  
    client_max_body_size 100M;  
  
    proxy_connect_timeout 60s;  
    proxy_send_timeout 300s;  
    proxy_read_timeout 300s;  
  
    proxy_buffering off;  
  
    location / {  
        proxy_pass http://192.168.31.17:3000;  
  
        proxy_http_version 1.1;  
  
        proxy_set_header Host $host;
```

```
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
    proxy_set_header X-Forwarded-Host $host;
    proxy_set_header X-Forwarded-Port 443;

    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}

location /loki/ {
    proxy_pass http://192.168.31.17:3100/loki/;

    proxy_http_version 1.1;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
    proxy_set_header X-Forwarded-Host $host;
    proxy_set_header X-Forwarded-Port 443;
}

location = /ready {
    proxy_pass http://192.168.31.17:3100/ready;

    proxy_http_version 1.1;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
}
}
```

38.5. Explicação da configuração

Bloco HTTP

```
server {
    listen 80;
    server_name logs.jmsalles.homelab.br;

    return 301 https://$host$request_uri;
}
```

Função:

Redireciona todo acesso HTTP para HTTPS.

Bloco HTTPS

```
server {  
    listen 443 ssl;  
    http2 on;  
  
    server_name logs.jmsalles.homelab.br;
```

Função:

Publica o domínio logs.jmsalles.homelab.br usando HTTPS.

Certificado

```
ssl_certificate      /etc/nginx/ssl/jmsalles.homelab.br.crt;  
ssl_certificate_key  /etc/nginx/ssl/jmsalles.homelab.br.key;
```

Função:

Usa o certificado wildcard/local que você já utiliza no Nginx do homelab.

Grafana

```
location / {  
    proxy_pass http://192.168.31.17:3000;  
}
```

Função:

Tudo que chegar em https://logs.jmsalles.homelab.br será enviado para o Grafana.

Loki

```
location /loki/ {  
    proxy_pass http://192.168.31.17:3100/loki/;  
}
```

Função:

Permite acessar endpoints do Loki usando:
<https://logs.jmsalles.homelab.br/loki/>

Exemplo:

<https://logs.jmsalles.homelab.br/loki/api/v1/labels>

será encaminhado para:

<http://192.168.31.17:3100/loki/api/v1/labels>

Health check do Loki

```
location = /ready {  
    proxy_pass http://192.168.31.17:3100/ready;  
}
```

Função:

Permite validar o Loki pelo Nginx usando:
<https://logs.jmsalles.homelab.br/ready>

38.6. Validar sintaxe do Nginx

Executar:

```
docker compose -f /home/jmsalles/nginx/docker-compose.yml exec nginx nginx -t
```

Resultado esperado:

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

38.7. Recarregar Nginx

Se o teste estiver correto:

```
docker compose -f /home/jmsalles/nginx/docker-compose.yml exec nginx nginx -s
reload
```

Validar container:

```
docker compose -f /home/jmsalles/nginx/docker-compose.yml ps
```

38.8. Testar acesso pelo Nginx

A partir de qualquer host da rede:

```
curl -Ik https://logs.jmsalles.homelab.br
```

Resultado esperado:

```
HTTP/2 200
```

ou:

```
HTTP/1.1 200 OK
```

Testar Grafana:

```
curl -I https://logs.jmsalles.homelab.br
```

Testar Loki pelo path **/ready**:

```
curl -k https://logs.jmsalles.homelab.br/ready
```

Resultado esperado:

```
ready
```

Testar labels do Loki:

```
curl -k "https://logs.jmsalles.homelab.br/loki/api/v1/labels" | jq
```

Resultado esperado:

```
{
  "status": "success",
  "data": [
    "env",
    "fqdn",
    "host",
    "job",
    "log_type",
    "role",
    "source"
  ]
}
```

38.9. Acessar no navegador

Acessar:

```
https://logs.jmsalles.homelab.br
```

Login:

```
Usuário: admin
Senha: 4796@@Eumesmo1234567
```

Depois validar no Grafana:

```
Explore  
Data source: Loki
```

Consulta:

```
{host="observabilidade"}
```

38.10. Ajustar datasource do Grafana se quiser usar URL pública

Por padrão, o datasource está configurado internamente assim:

```
url: http://loki:3100
```

Esse é o melhor modelo, porque o Grafana fala com o Loki pela rede Docker interna.

Não é necessário alterar.

Mas, se quiser testar usando a URL publicada pelo Nginx, o datasource poderia ser:

```
url: https://logs.jmsalles.homelab.br
```

Nesse caso, o endpoint `/loki/` precisaria ser usado corretamente pelo Grafana, então **não recomendo alterar agora**.

Modelo recomendado:

```
Grafana -> Loki pela rede Docker interna: http://loki:3100  
Usuário -> Grafana via Nginx: https://logs.jmsalles.homelab.br  
Alloy -> Loki direto na VM: http://192.168.31.17:3100/loki/api/v1/push
```

38.11. Ajustar configuração de agentes Alloy futuros

Para os próximos hosts Linux/Docker/Kubernetes, usar como destino:

```
http://192.168.31.17:3100/loki/api/v1/push
```

Exemplo no Alloy:

```
loki.write "default" {  
  endpoint {  
    url = "http://192.168.31.17:3100/loki/api/v1/push"  
  }  
}
```

Evite usar o Nginx para ingestão de logs em massa:

```
https://logs.jmsalles.homelab.br/loki/api/v1/push
```

Motivo:

```
Menos camada no caminho  
Menor risco de timeout  
Menor chance de problema com proxy  
Melhor desempenho para envio contínuo de logs
```

39. Troubleshooting

39.1. Container Loki não sobe

Ver logs:

```
cd /opt/observability  
docker compose logs loki
```

Validar configuração:

```
cat /opt/observability/loki/loki-config.yml
```

Validar permissão:

```
ls -lah /opt/observability/data/loki
```

Ajustar permissão:

```
chown -R 10001:10001 /opt/observability/data/loki
```

Subir novamente:

```
docker compose up -d
```

39.2. Grafana não abre direto pela porta 3000

Ver container:

```
docker ps | grep grafana
```

Ver logs:

```
cd /opt/observability  
docker compose logs grafana
```

Validar porta:

```
ss -lntp | grep 3000
```

Testar localmente:

```
curl -I http://127.0.0.1:3000
```

Testar via IP:

```
curl -I http://192.168.31.17:3000
```

39.3. Grafana não abre pelo Nginx

Validar DNS:

```
getent hosts logs.jmsalles.homelab.br
```

Resultado esperado:

```
192.168.31.37 logs.jmsalles.homelab.br
```

Validar Nginx:

```
docker compose -f /home/jmsalles/nginx/docker-compose.yml exec nginx nginx -t
```

Ver logs do Nginx:

```
docker compose -f /home/jmsalles/nginx/docker-compose.yml logs -f nginx
```

Testar acesso do host Nginx para a VM:

```
curl -I http://192.168.31.17:3000
```

Testar acesso HTTPS:

```
curl -Ik https://logs.jmsalles.homelab.br
```

39.4. Loki não responde **ready**

Ver logs:

```
cd /opt/observability  
docker compose logs loki
```

Testar porta:

```
ss -lntp | grep 3100
```

Testar endpoint:

```
curl -v http://127.0.0.1:3100/ready
```

Reiniciar Loki:

```
cd /opt/observability  
docker compose restart loki
```

39.5. Loki funciona direto, mas não pelo Nginx

Testar direto:

```
curl -s http://192.168.31.17:3100/ready
```

Testar pelo Nginx:

```
curl -k https://logs.jmsalles.homelab.br/ready
```

Testar labels direto:

```
curl -s "http://192.168.31.17:3100/loki/api/v1/labels" | jq
```

Testar labels pelo Nginx:

```
curl -k "https://logs.jmsalles.homelab.br/loki/api/v1/labels" | jq
```

Se direto funciona e pelo Nginx não, revisar este bloco:

```
location /loki/ {  
    proxy_pass http://192.168.31.17:3100/loki/;  
}
```

Depois testar Nginx:

```
docker compose -f /home/jmsalles/nginx/docker-compose.yml exec nginx nginx -t
```

Recarregar:

```
docker compose -f /home/jmsalles/nginx/docker-compose.yml exec nginx nginx -s reload
```

39.6. Grafana abre, mas não encontra logs

Gerar log de teste:

```
logger "TESTE MANUAL LOKI $(date)"
```

Verificar se `/var/log/messages` recebeu:

```
tail -n 20 /var/log/messages
```

Ver logs do Alloy:

```
cd /opt/observability  
docker compose logs alloy-local
```

Validar labels no Loki:

```
curl -s "http://127.0.0.1:3100/loki/api/v1/labels" | jq
```

Consultar no Grafana:

```
{host="observabilidade"}
```

39.7. Alloy não coleta `/var/log/secure`

Verificar se o arquivo existe:

```
ls -lh /var/log/secure
```

Verificar se há conteúdo:

```
tail -n 20 /var/log/secure
```

Ver logs do Alloy:

```
cd /opt/observability  
docker compose logs alloy-local
```

39.8. DNS não resolve

Validar:

```
getent hosts logs.jmsalles.homelab.br
```

Validar `/etc/resolv.conf`:

```
cat /etc/resolv.conf
```

Testar com DNS interno:

```
nslookup logs.jmsalles.homelab.br 192.168.31.24
```

ou:

```
dig @192.168.31.24 logs.jmsalles.homelab.br
```

Enquanto o DNS não estiver pronto, usar direto:

```
http://192.168.31.17:3000
```

40. Próximas fases recomendadas

Depois de validar esta primeira etapa, seguir nesta ordem:

```
Fase 2 - Coletar logs do docker-hml / Portainer  
Fase 3 - Coletar logs dos servidores Linux  
Fase 4 - Coletar logs do Kubernetes  
Fase 5 - Criar dashboards iniciais  
Fase 6 - Criar alertas básicos no Grafana  
Fase 7 - Integrar visão operacional com Zabbix
```

Minha recomendação para o próximo tutorial é começar pelo host:

```
docker-hml.jmsalles.homelab.br
```

Coletando logs de:

```
Portainer  
Nginx reverse proxy  
Gitea  
Nexus  
Vault  
MinIO  
phpIPAM  
Stirling PDF  
Docker registry
```
